Data Foundations & Readiness

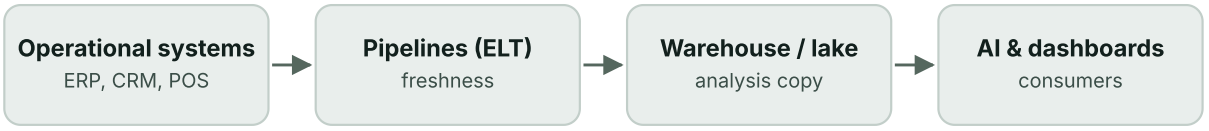
1. Why AI projects die

Post-mortems converge on the same cause: not algorithms but **data**. Data that doesn't exist, can't be accessed, can't be joined, can't be trusted, or can't legally be used. The discipline is sequencing: data readiness first, ambition second.

2. The data estate

- **Operational systems** (ERP, CRM, POS): where data is born, built for transactions, not analysis
- **Warehouse** (BigQuery, Snowflake): structured, governed copy for analysis · **Lake**: cheap raw storage, can become a swamp · **Lakehouse**: both
- **Pipelines** (ELT plus orchestration): where freshness is won or lost
- Ask: where does it live, how fresh is it, who owns it, can we query it without touching production?

✎ DIAGRAM



Data is born in operational systems and copied into an analytical layer; AI should read from the copy, not production.

3. Quality: six testable dimensions

- **Accuracy · completeness · consistency · timeliness · uniqueness · validity**
- Each is measurable, so each can be an automated test
- **Data contract**: an enforced agreement on schema, meaning, freshness and quality, so an upstream team can't silently break a model by renaming a column

4. The data-readiness assessment

A 1 to 3 week fixed-fee diagnostic, run before (or as phase zero of) any AI engagement. Score each candidate use case red / amber / green on:

1. **Existence & history**: captured at the right grain, far enough back?
2. **Quality**: profile it for a day; it reveals more than a week of meetings
3. **Accessibility**: a queryable copy, or locked in a vendor silo?

4 **Joinability**: one customer ID across systems, or three? (identity resolution can be a hidden multi-month project)

5 **Rights & permissions**: may it legally be used for this purpose?

✎ DIAGRAM

Use case	Exists	Quality	Access	Joins	Rights
Churn model	Green	Amber	Green	Red	Green
Support RAG	Green	Green	Amber	Green	Amber

Example readiness scorecard. Greens proceed, ambers get scoped fixes, reds get honest bad news early.

5. The permissions trap

> ⚠ A RAG assistant that indexes every document and answers anyone can leak documents to people who were never allowed to see them. Retrieval must respect each user's permissions; raising this early is a credibility moment.

6. Red flags in the room

- 'It's all in the warehouse' with no specifics · finance and marketing quoting different revenue numbers (no semantic layer) · key data in a vendor tool with no API · nobody owns the pipeline

7. Text-to-SQL: talking to your data

The most requested 'AI on our data' use case, and easy to get confidently wrong. The hard part isn't SQL, it's **meaning**.

1 User asks a question

2 **Retrieve the relevant schema** (a few tables and columns with descriptions), not the whole warehouse

3 LLM writes SQL, guided by example question → SQL pairs

4 **Validate**: parses, read-only, allowed tables only, LIMIT, dry-run cost check

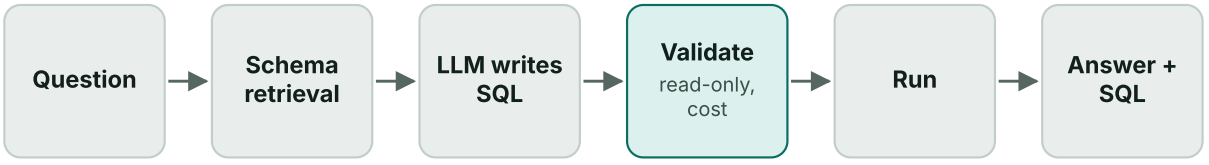
5 Run it

6 Answer in words and **show the SQL and the numbers**

- Fails on: ambiguous metrics ('revenue': gross or net?), wrong joins that duplicate rows, missing filters (test accounts), expensive scans

- Fixes: a **semantic layer** (one definition per metric), clarifying questions, a read-only account with row-level security
- Evaluate with **execution accuracy**: does the query return the right result? Never compare SQL text

✎ DIAGRAM



Validation before execution is what makes text-to-SQL safe to put in front of business users.

SAY IT

"Before scoping the AI, I'd run a short data-readiness check on each use case, and for text-to-SQL I'd put a semantic layer underneath, because the hard part is agreeing what revenue means."